In Brightest Way, In Blackest Miss

Lab 10 · Double digit labs; I feel old junior.

Buffer Slide

- Doubts from last week's lab? Ask them now.
- Outlab: status and doubts (TA update).
- The last hour is the lab exam based on lab 9 content.

Insert announcements.


Insert announcements.
JUNIOR

Plan for this lab

- Revise Lab 9
- Cover some new content
  - Revise logic?
- Lab exam (easy if you pay attention)

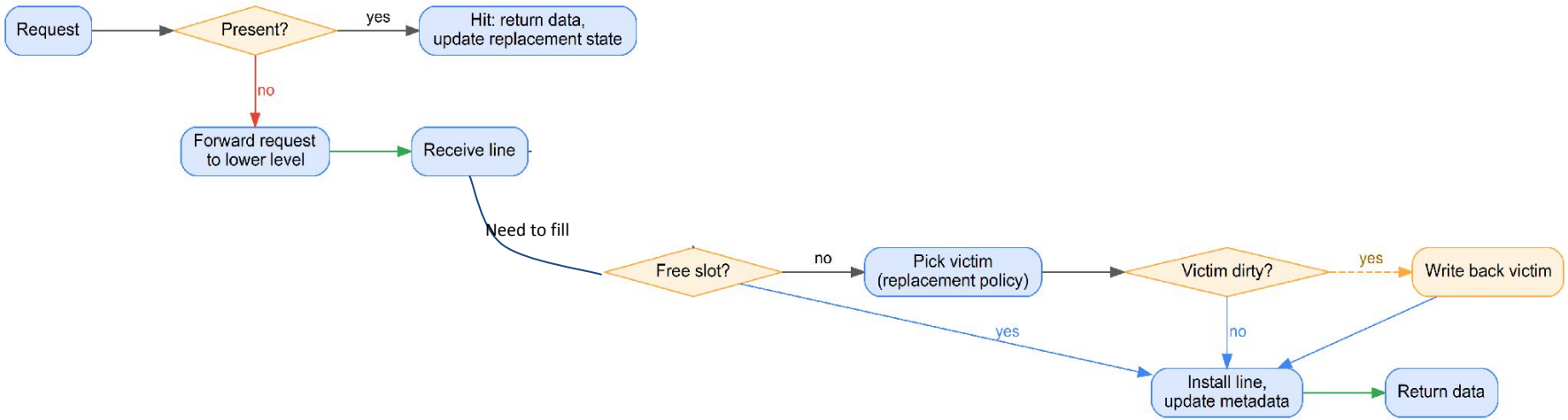
Do you want revision or new content first?

Part N

Cache inclusion policies ft. logic

- What one cache does on an access
- Two caches: L1 and L2
- Inclusive, exclusive and NINE
- Building an exclusive hierarchy

What happens on a cache access?

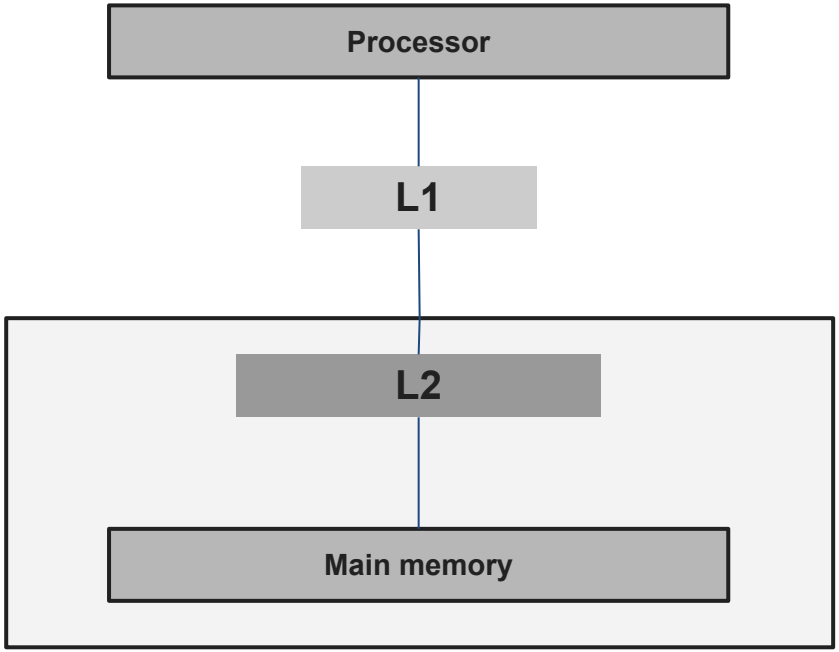


Something new to think about: Think of the cache hierarchy as a set of axioms defined over cache lines. The implementation then is a program to enforce these.

One of the conclusions/theorems from the program is the NINE policy of ChampSim (more later)... We can edit the axioms to change this to inclusive/exclusive.

The slides have these axioms (optional).

A cache of a cache



Hierarchy of cache is nothing but a cache for the later levels.

L2, was made to cache the requests to Main memory.

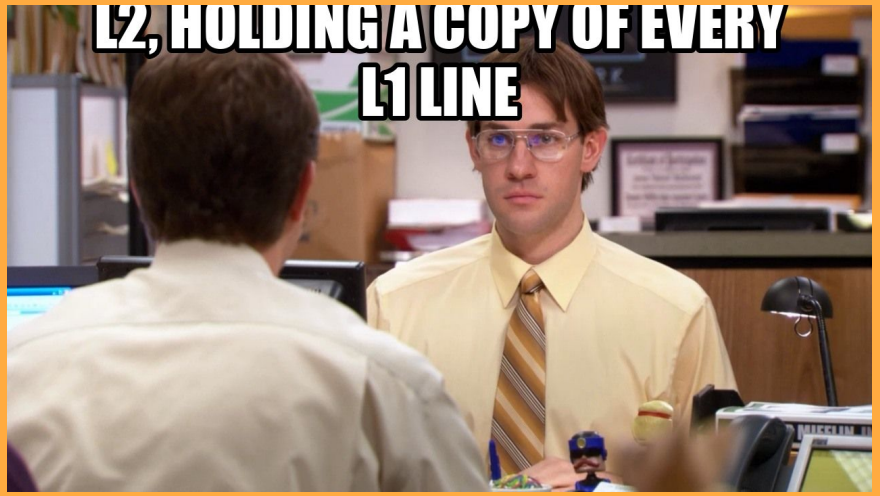
L1, was made to cache the requests to L2.

The above view for hierarchy is a very simplistic view that helps in abstracting out other details of hierarchy.

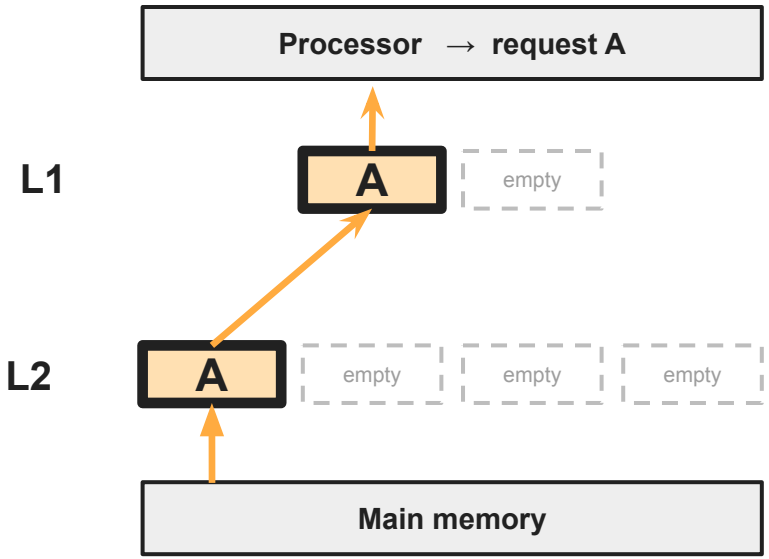
Q: Is this view good enough? Are there any in-efficiencies?

Inclusive

Every line in L1 is also in L2



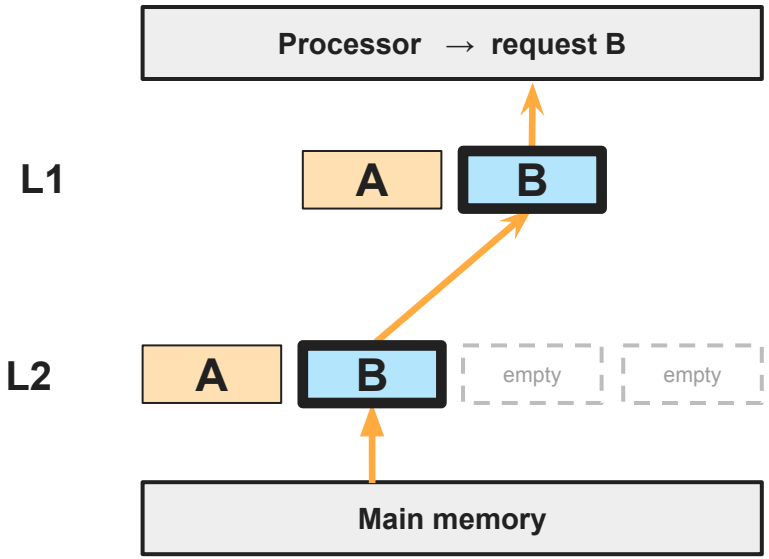
Why does L1 look like a subset of L2? (1/4)



Request A

1. L1 misses. L2 misses.
2. A comes from memory.
3. A is installed in L2 and in L1.

Why does L1 look like a subset of L2? (2/4)

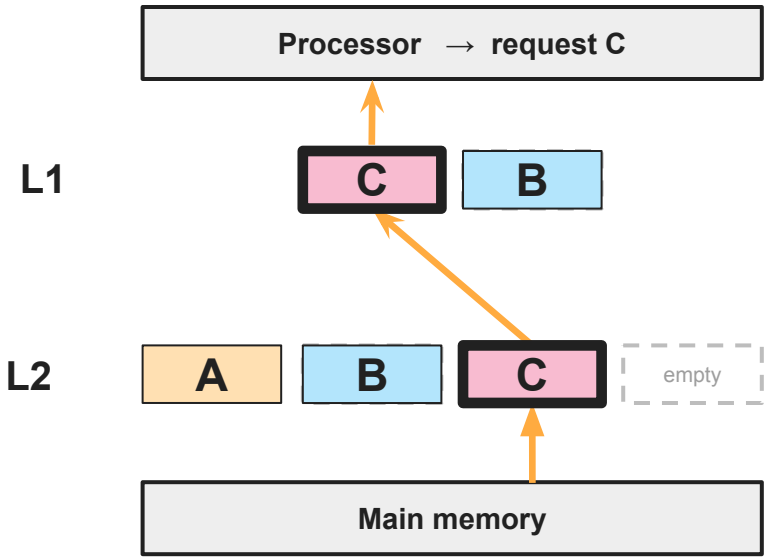


Request B

1. L1 misses. L2 misses.
2. B comes from memory.
3. B is installed in both.

L1 is now full.

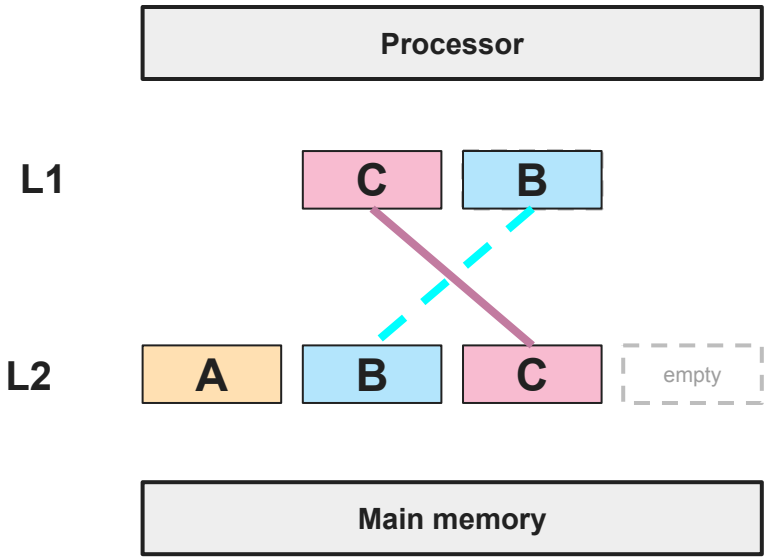
Why does L1 look like a subset of L2? (3/4)



Request C

1. L1 misses. L2 misses.
2. C comes from memory and is installed in L2.
3. L1 is full: it evicts A, its least recently used line.
4. A is clean, so no writeback.
5. C is installed in L1. L2 still has room, so it keeps A.

Why does L1 look like a subset of L2? (3/4)



Final State

With the simple policy we had, we always end up with L1's entries being subset of L2's entries.

Q: Is this helpful? Or is it harmful?

Inclusive, exclusive and NINE hierarchies (LOGIC???)

Inclusive

Every line in L1 is also in L2. Part of L2 holds copies of L1's lines.

$INC\ X : X \in L1 \rightarrow X \in L2$

Exclusive

A line is in L1 or in L2, never in both. No copies, so L1 + L2 hold more different lines.

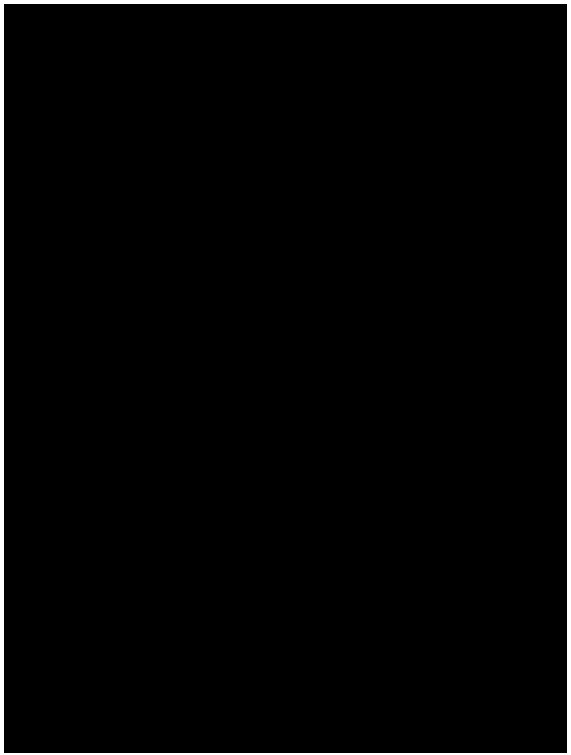
$EXC\ X : X \in L1 \rightarrow \neg(X \in L2)$

$ie: \neg((X \in L1) \& (X \in L2))$

NINE

No rule. A line can be in L1 only, in L2 only, or in both.

$NINE\ X : T$

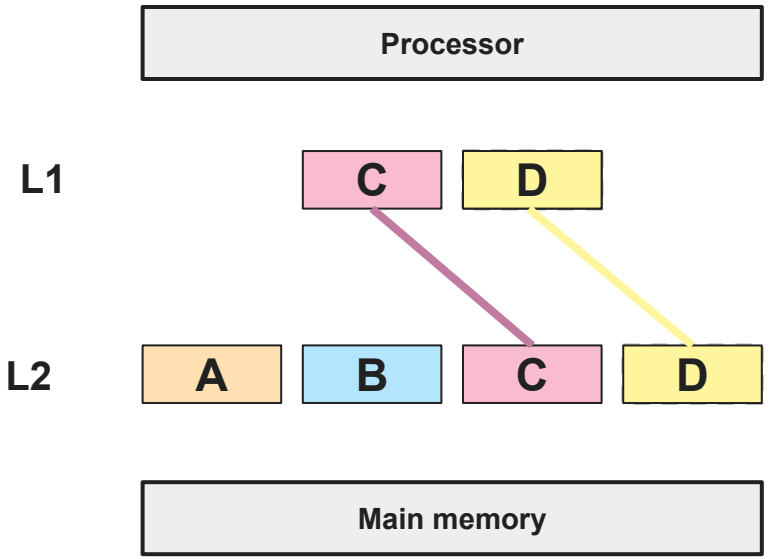


Exclusive

A line is in L1 or in L2, never in both



The cost of duplicate copies



Duplicate copies use L2 capacity.
We could have stored other useful data.

After requests A, B, C, D

Both levels allocate every demand line.

L1 keeps the 2 most recent: C, D.

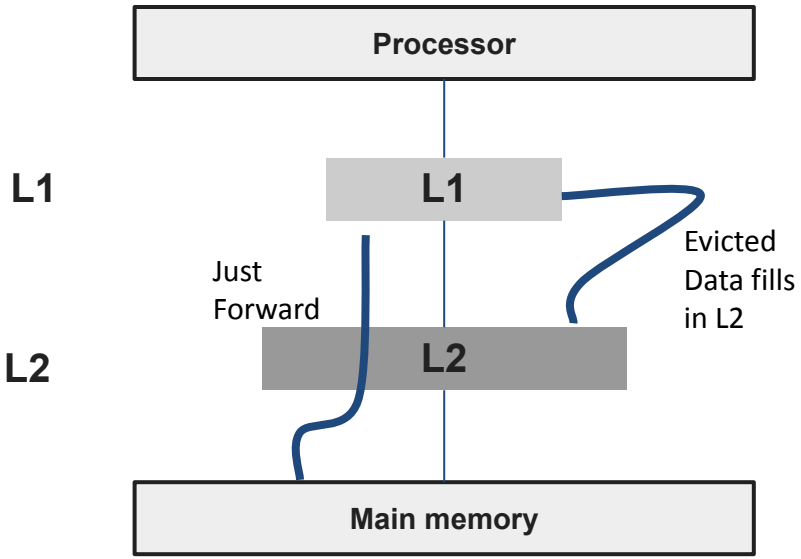
L2 keeps all 4: A, B, C, D.

Two of L2's four lines copy L1.

Q: If a line is already in L1, when might L2 need to access that line again?

H: Think about what happens if L1 evicts the line and the processor asks for it later.

Can we have an Exclusive Policy?



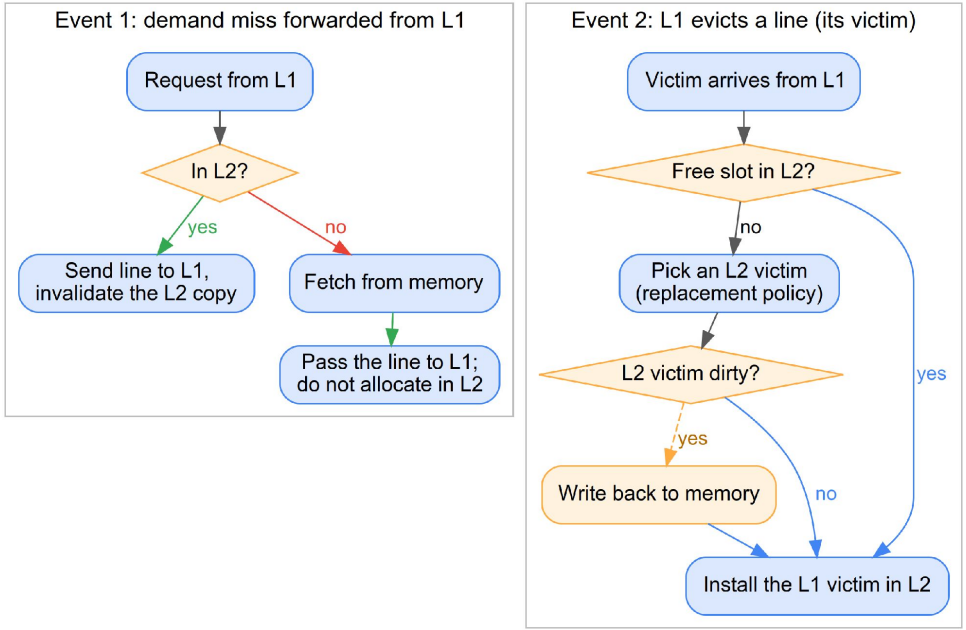
What should we do on a request at L2?

Given that L1 caches the data L2 will remove the data from its entries, as it will already be present in L1.

If we forward at L2, when will we fill it?

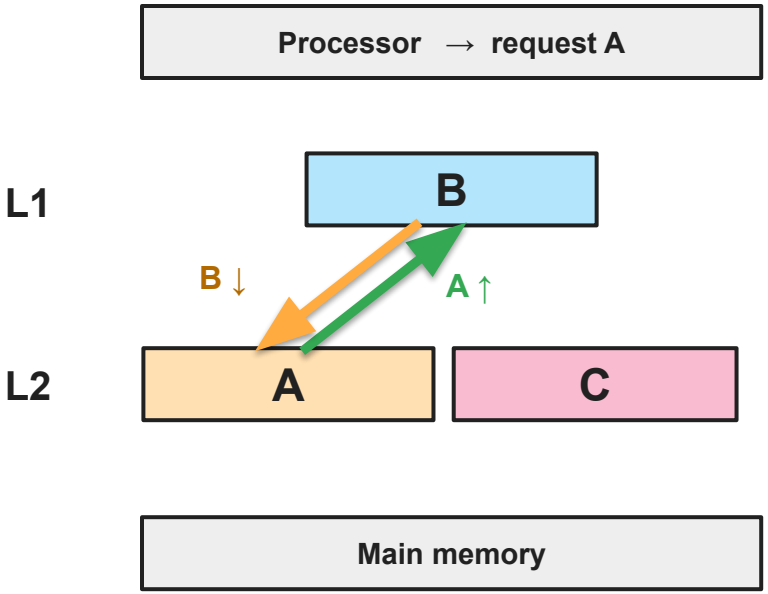
We can fill the L2 with victim of L1 cache, so that they can be reused when needed.

Finalized Exclusive Policy



An L1 eviction becomes a chance to keep the displaced line in L2.

An L1 miss can exchange lines with L2 (1/2)

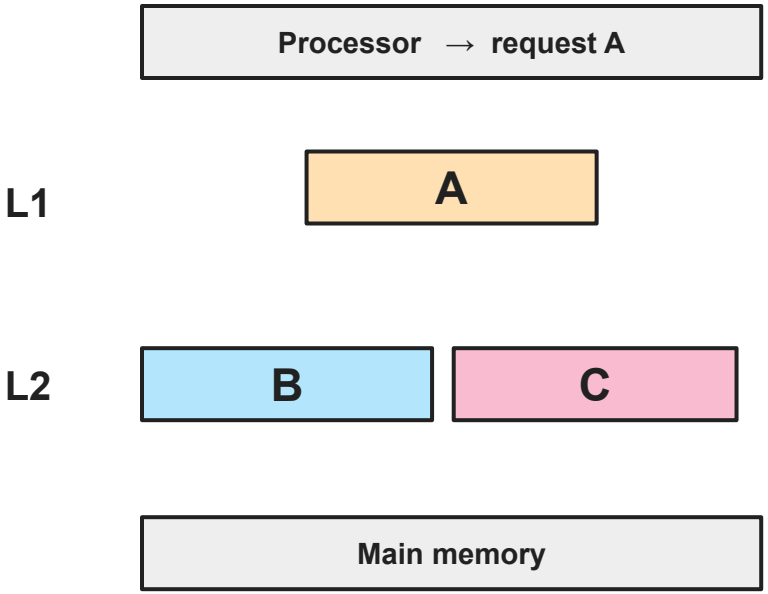


Request A. L1: B L2: A, C

1. L1 misses.
2. L2 hits on A.
3. A moves from L2 to L1.
4. L1's victim B moves from L1 to L2.

These two moves are a **swap**.

An L1 miss can exchange lines with L2 (2/2)



Final state. L1: A L2: B, C

No block is in both caches.

If L2 had missed: fetch A from memory, install it in L1, move L1's victim to L2, and apply L2's normal replacement and writeback rules.

Q: Why does an L2 hit need more than copying A into L1?

H: A copy of A would stay in L2 unless it is removed or exchanged.

NINE

Non-inclusive, non-exclusive: copies allowed, not required



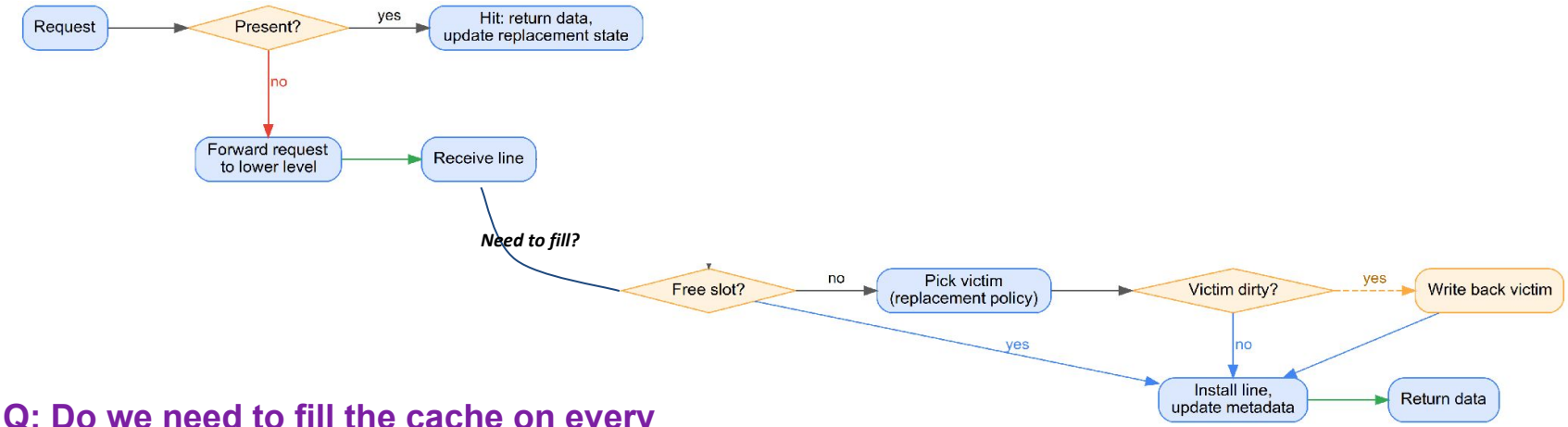
Before Understanding NINE, let’s introduce a new concept called ‘Bypassing’.

Part N+1

Bypassing

- A miss does not have to become a fill
- Independent bypassing breaks inclusion
- Comparing the three policies

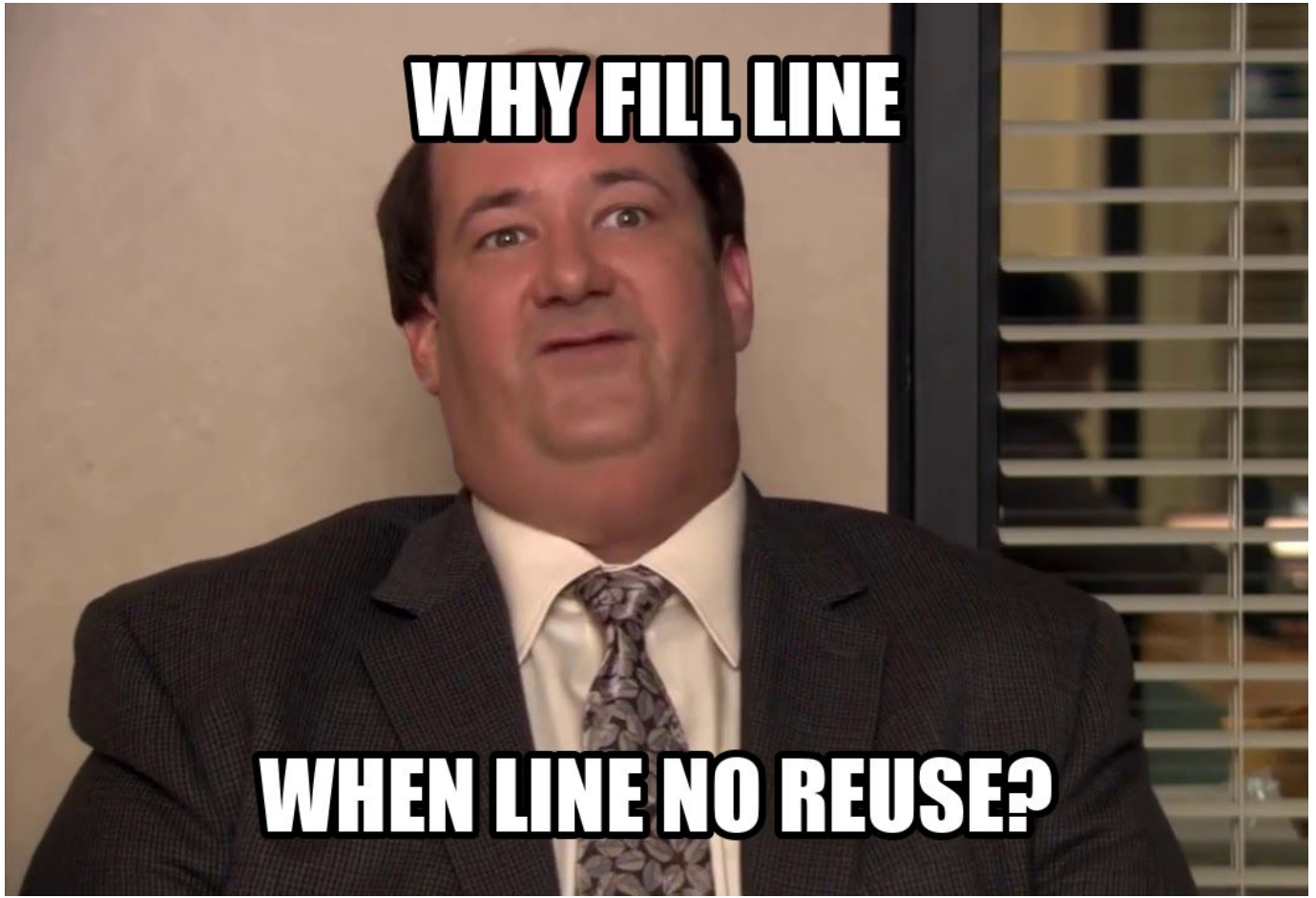
A miss does not have to become a fill



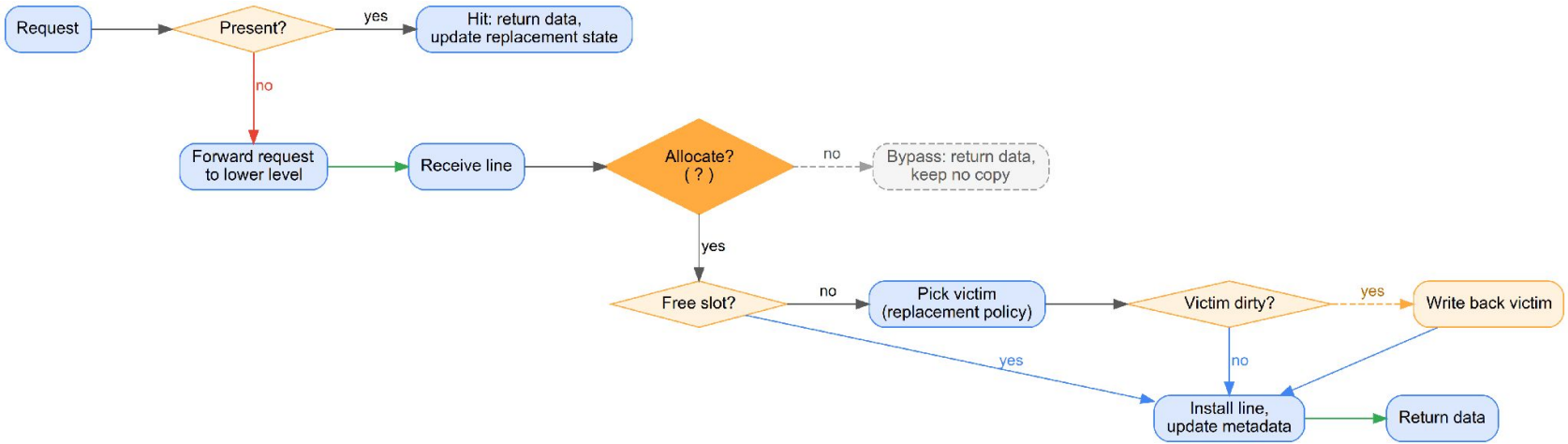
Q: Do we need to fill the cache on every miss?

Suppose a not going to be reused, do we need to fill it in the cache?

Not filling lines in the cache even on a demand is called **bypass**.


WHY FILL LINE
WHEN LINE NO REUSE?

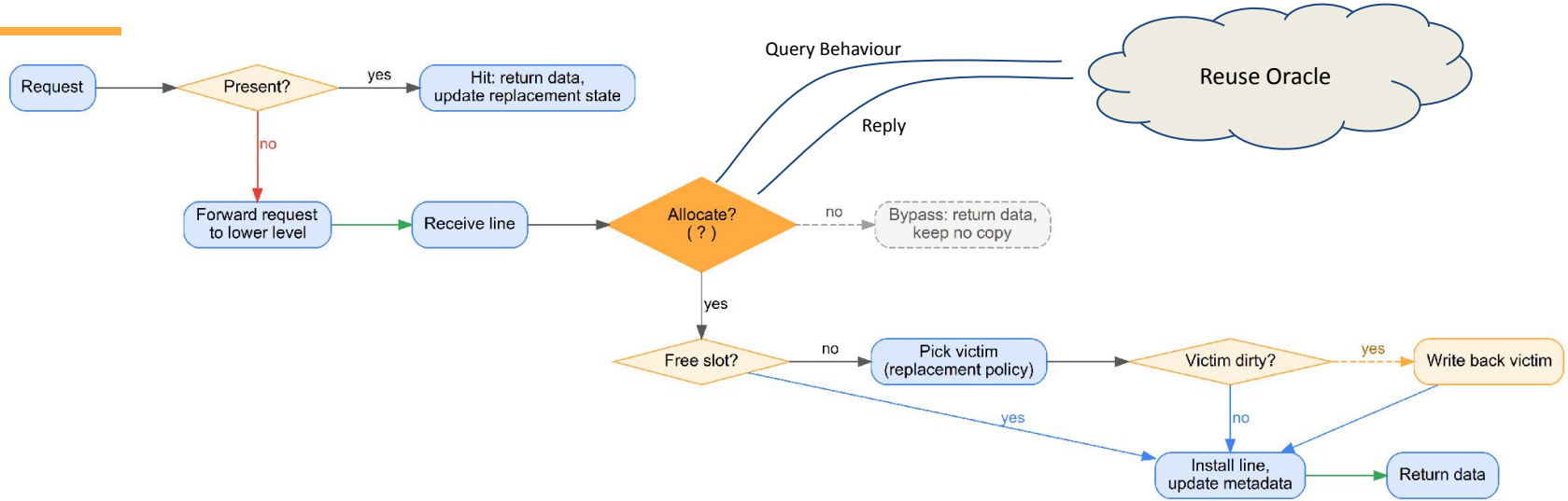
A miss does not have to become a fill



- Some lines will not be reused, so there is no point filling them in the L2 or L3. Not filling them is called **bypass**.
- **Allocate**: install the line, evicting a resident line if needed.
- **Bypass**: deliver the data to the requester, keep no copy here.

Q: How do we know to Bypass or not?

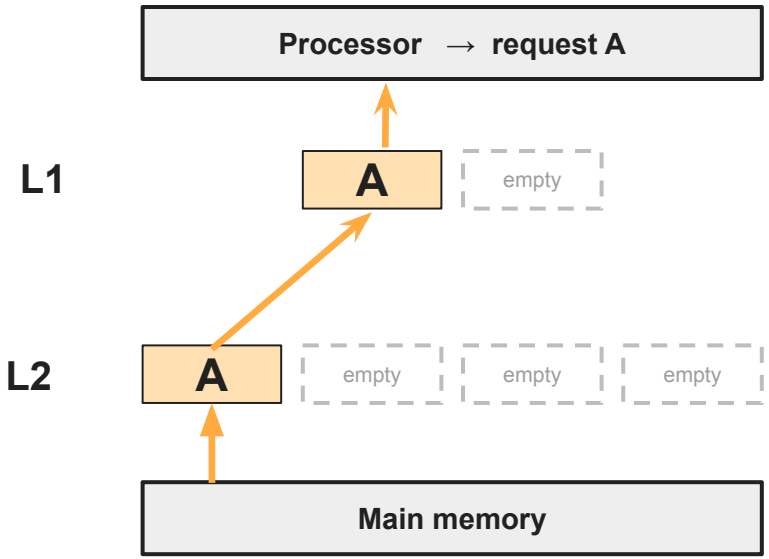
Dead Block Prediction



• **Reuse Oracle :** An algorithm that guess the reuse behaviour i.e, whether it will be used or is it dead?.

We query the reuse oracle about the Behaviour of particular line, The oracle predicts if it will be useful using some heuristics. (Which we will design later), for now treat it as an abstract all knowing being.

L1 and L2 need not store the same lines (1/2)

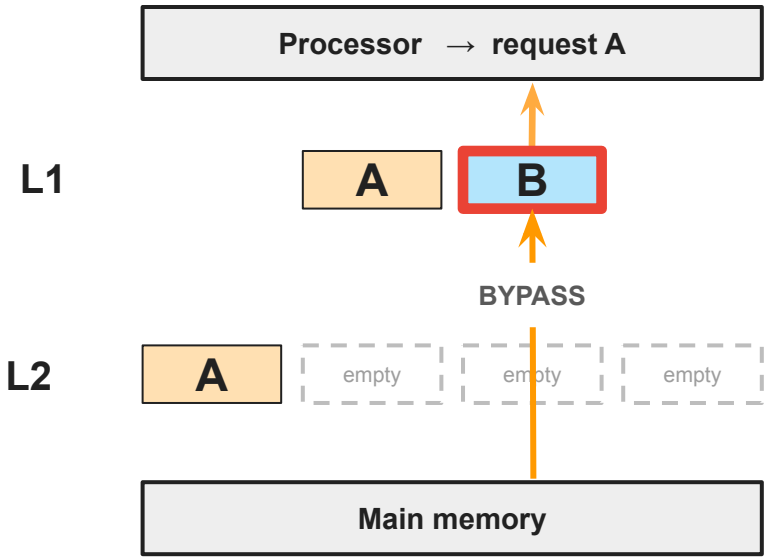


Request A (caches start empty)

1. L1 misses. L2 misses.
2. Memory returns A.
3. L2 chooses to allocate A.
4. L1 chooses to allocate A.

A is in both: still consistent with inclusion.

L1 and L2 need not store the same lines (2/2)



Request B

1. L1 misses. L2 misses.
2. Memory returns B.
3. L2 chooses to **bypass** B.
4. L1 chooses to allocate B.

$C_1 = \{A, B\}$, $C_2 = \{A\}$

$C_1 \not\subset C_2$ and $C_1 \cap C_2 = \{A\}$: a NINE state.

Independent bypassing can create an L1-only line, so inclusion is no longer guaranteed.

Which relationship should a hierarchy enforce?

Property	Inclusive	Exclusive	NINE
`Relationship`	$C_1 \subseteq C_2$	$C_1 \cap C_2 = \varnothing$	No restrictions
`Duplicate lines?`	Yes	No	Yes
`L1-only lines?`	No	Yes	Yes
`L2-only lines?`	Yes	Yes	Yes
`Coordination`	to keep inclusion	to keep exclusivity	depends on the design
`Capacity`	duplicates use space	more unique lines	flexible overlap
`Main challenge`	keeping inclusion; duplicates	moving and swapping lines	good allocation and replacement policies

Three relationships, three trade-offs

- **Inclusive**: a simple subset rule, but duplicate copies use capacity.
- **Exclusive**: no duplicates, but lines must move between levels.
- **NINE**: overlap is allowed. Allocation and replacement decide the rest, which gives freedom and is easy to build.
- An inclusion policy describes contents. Performance comes from how allocation, replacement and data movement are built.
- **Note : The exact details we explained are ‘naturally occurring example policies’ Any policy can be studied in terms of Inclusivity relation.**

Q: If each level could decide on its own whether to keep a line, what would help it decide?

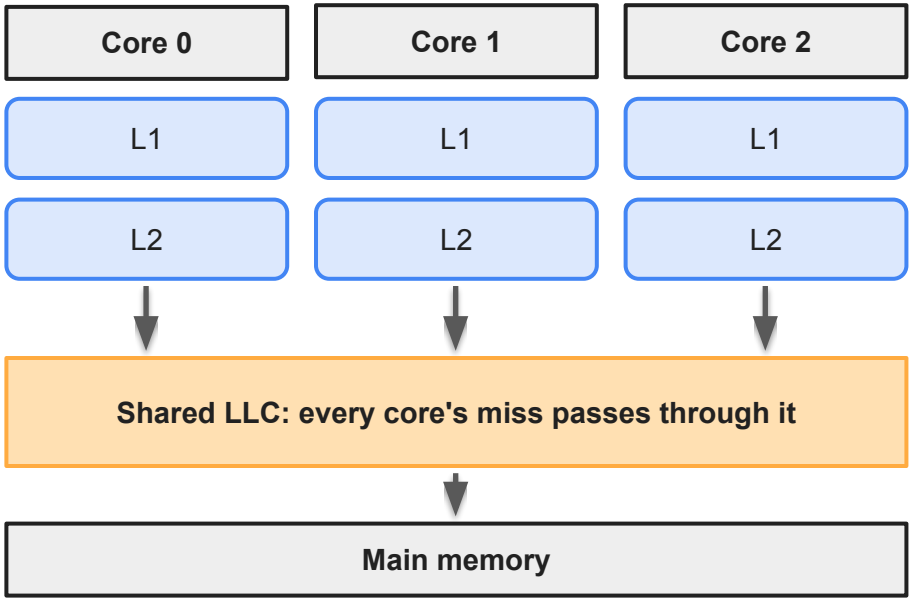
H: Reuse probability, miss cost, capacity pressure, and the cost of moving or evicting a line.

Part N+2

Real hierarchies

- Which processors use which policy
- Why a shared LLC is often inclusive

Why a shared LLC is often inclusive – Cache Coherence*



The shared LLC is a natural place to answer: does anyone else hold this line?

- A core misses on X. Does another core hold X?
- **Inclusive LLC**: an LLC miss means no core has X. Go to memory.
- **Non-inclusive LLC**: needs a separate directory of the cores' lines.
- **Cost**: an LLC eviction must invalidate every copy above.

Q: Core 1 keeps hitting on X in its L2, so the LLC never sees X used. Under inclusion, what happens when the LLC evicts X?

H: The LLC copy cannot simply go away on its own.

* may be a common interview question topic idx ¯_(ツ)_/¯

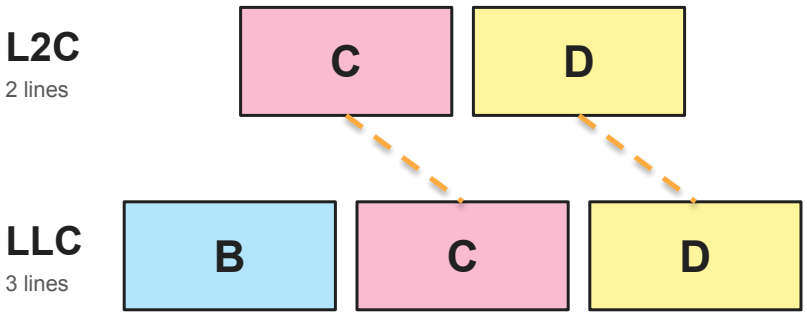
Part N+3

An exclusive LLC in ChampSim

- Copies waste LLC space
- Three moves: bypass, victim down, hit up
- Four steps in cache.cc
- Results and questions

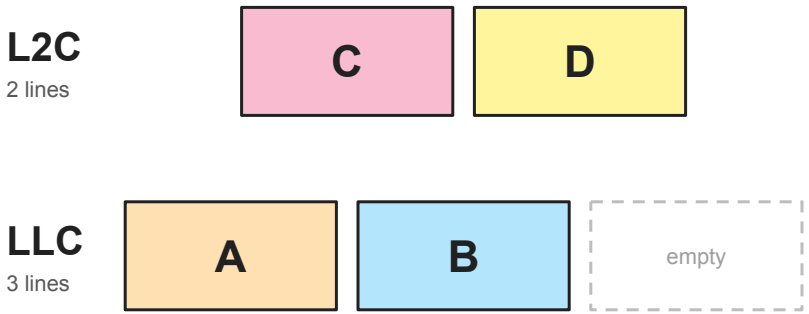
Our ChampSim stores most lines twice

Today (NINE)



3 different lines

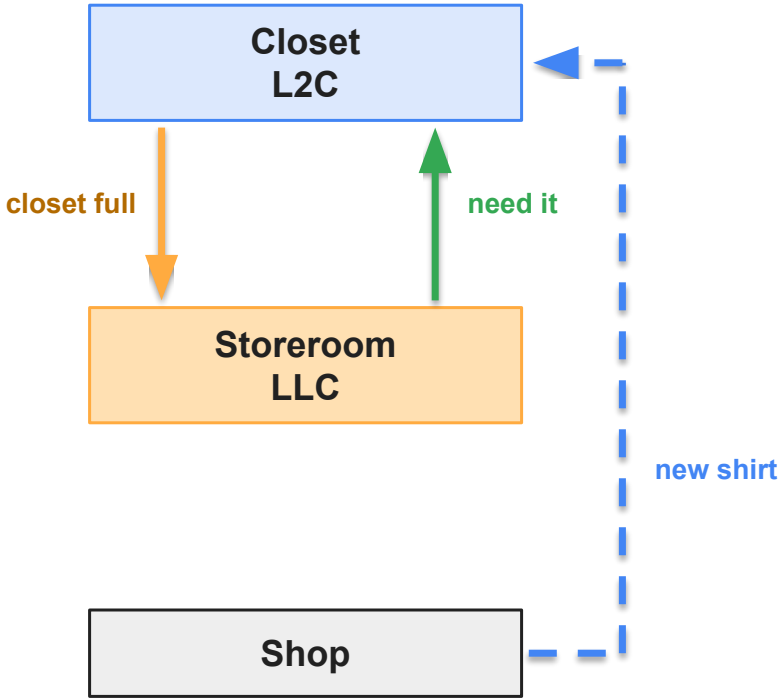
Exclusive LLC



4 different lines

In ChampSim today, 8191 of the 8192 L2C lines are also in the LLC.

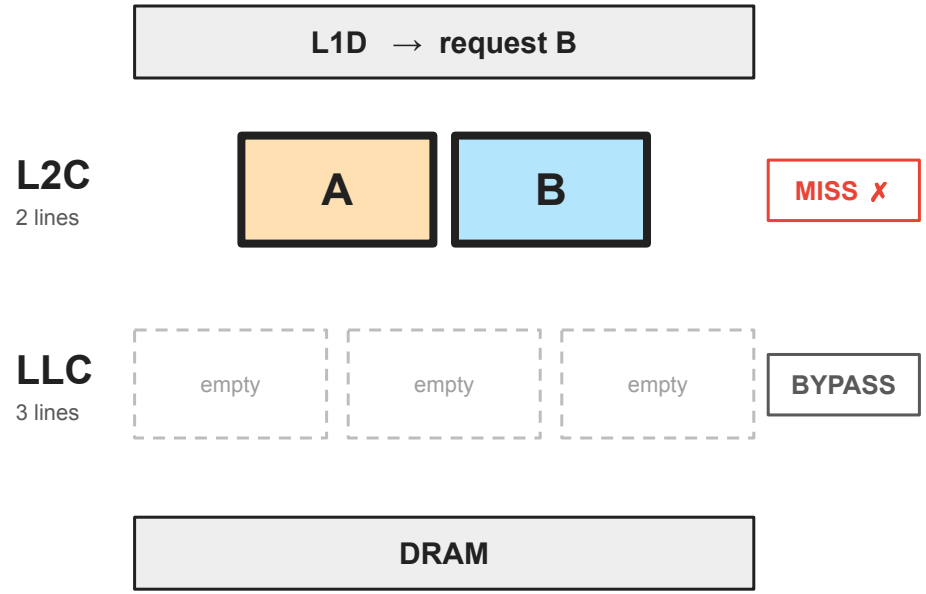
An analogy: a closet and a storeroom



- A new shirt goes **straight to the closet**.
- A full closet sends its oldest shirt **to the storeroom**.
- A shirt you need **leaves the storeroom**.
- Each shirt is in one place, so both rooms together hold more shirts.

Kehno ko NINE but harkaton se Inclusive hi hai

Move 1: a line from DRAM skips the LLC

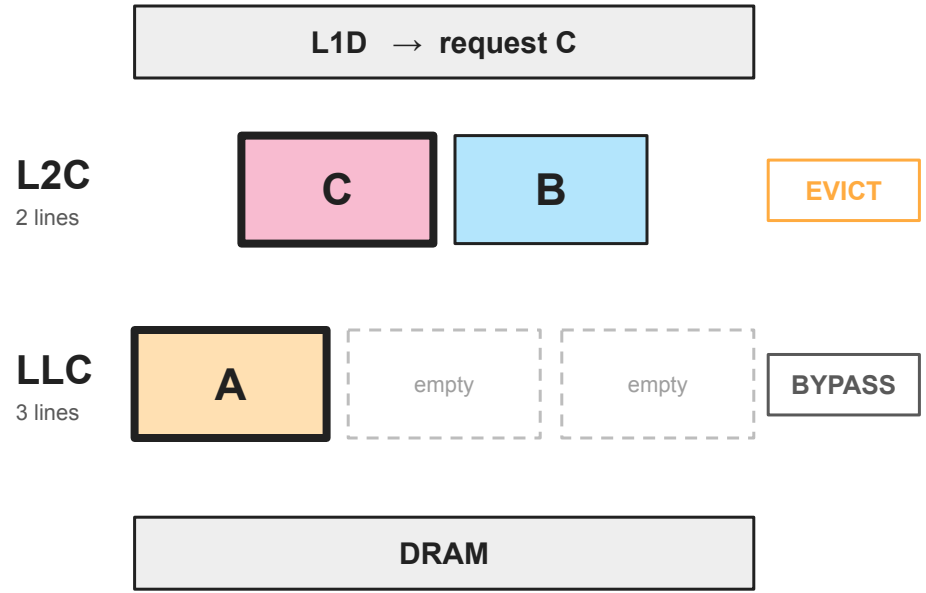


Requests A, then B

1. The L2C and the LLC miss.
2. DRAM returns the line.
3. The LLC **bypasses** it: it passes the data up and keeps no copy.
4. The L2C installs it.

Bypass = deliver the data, allocate nothing.

Move 2: an L2C victim moves down

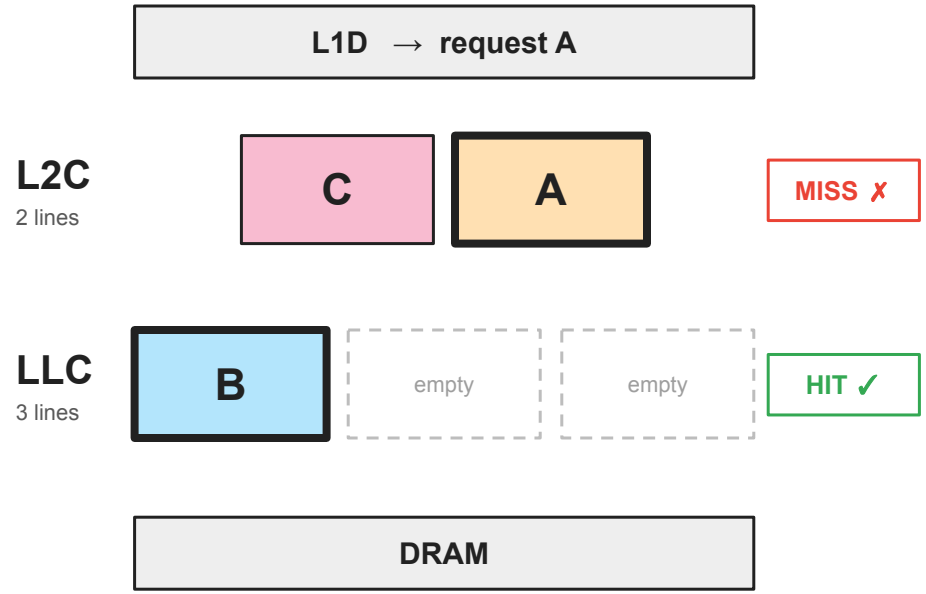


Request C

1. The L2C and the LLC miss.
2. The L2C is full: it evicts A.
3. A **moves down** to the LLC, clean or not.
4. C comes from DRAM and bypasses the LLC.

The LLC holds only L2C victims: a victim cache.

Move 3: an LLC hit moves the line up

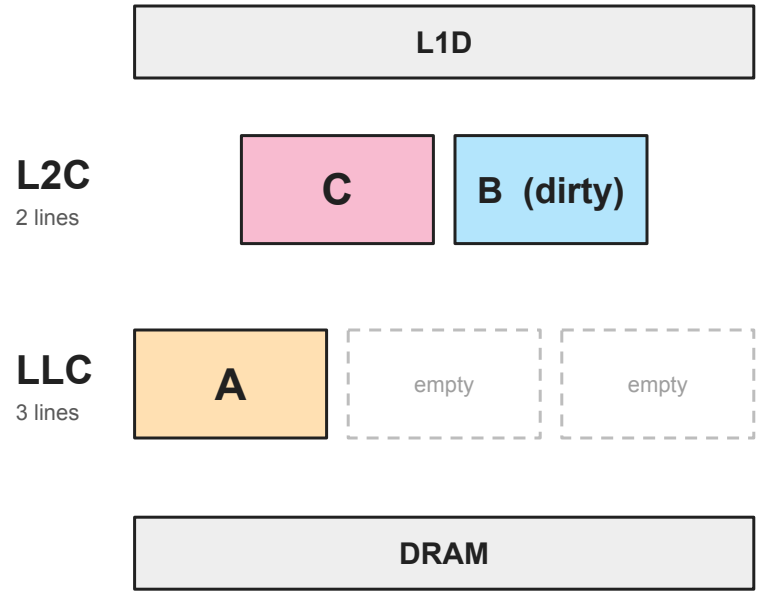


Request A

1. The L2C misses.
2. The LLC hits on A.
3. A **moves up**: the LLC drops its copy.
4. The L2C evicts B, and B moves down.

An LLC hit is a swap: A up, B down.

A dirty line stays dirty when it moves



The program wrote to B, so B is dirty.

B moves down: the LLC stores it **dirty**.

B moves up: the L2C stores it **dirty**.

A clean line stays **clean**.

The dirty bit travels with the line.

Q: What goes wrong if B arrives marked clean?

H: Follow B until it leaves the caches.

Optional: why a patch?

Optional

- An exclusive LLC moves lines up and down. The dirty bit must move with them.
- Given ChampSim cannot do this: a packet has no dirty bit, and every `WRITEBACK` counts as dirty. But… we wanna allow clean writebacks too.
- `task3.patch` adds a dirty bit to each packet. On its own it changes no numbers.

```
make task3                                  # patches a copy
./task3/apply_patch.sh ~/Desktop/ChampSim   # or by hand
```

Q: What did ChampSim not send dirty bit in the response/handle_fill path? Can they satisfy their cache axions without it?

Think about where you would need to send dirty bit up. Then think some more, do you really need it.

Optional: Axioms of the patch

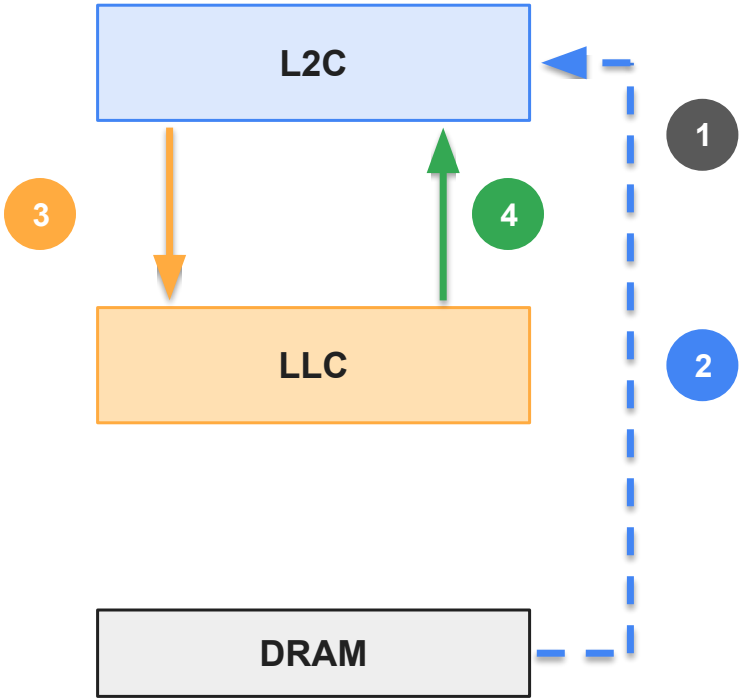
LOGIC?

0th axiom: Packet must contain a `dirty` bit. Blocks/lines already do.

1. Dirty bit must be sent in the packet on any up/down path.
2. Packet's dirty bit must be respected when filling in the line
3. When sending dirty line to an upper cache, mark `dirty=false` in it (Why?)

This is what the patch implements. See if you can find any violations.

Four steps in cache.cc



1. A bypass still sends its data up. (This is an artifact of champsim implementation not really a unique idea)

2. The LLC bypasses fills from DRAM.

3. The L2C sends every victim down.

4. An LLC hit moves the line up, dirty bit and all.

Enclose every change in `if (EXCLUSIVE_LLC)`.

`NAME` picks the cache: `LLC` or `cpu0_L2C`.

Optional: Axioms of EXCLUSIVE_LLC

LOGIC?

Assumes the patch axioms exist. These apply on top (some override). The idea is that using this set you can “derive” the ***¬(L2_x & LL_x)*** exclusivity condition as a theorem.

1. Any Dirty (D)/Clean (C) eviction from L2C must be written into LLC with same dirty bit.
2. Miss in L2C + hit in LLC must be filled in L2C with same dirty bit and evicted from LLC.
3. Miss in L2C + miss in LLC must be filled in L2C directly from DRAM i.e. bypassed in LLC.

Are these sufficient? Are they too strong?

See if you can find a violation in either the implementation of these axioms in the code, or in axioms themselves.*

NOTE 1: There is an easy trick where you dont need a dirty bit; just treat every line as dirty on an L2C eviction does it satisfy all constraints? Perhaps. What’s the downside?

* yes

Step 0: a bypass must still send the data up

```
// handle_fill()
if (way ≠ NUM_WAY) {
  ...   // install, then send data up
}
if (EXCLUSIVE_LLC && way == NUM_WAY) {
  for (auto ret : fill_mshr→to_return)
    ret→return_data(&(*fill_mshr));
}

// filllike_miss()
if (!(EXCLUSIVE_LLC && bypass))
  impl_replacement_update_state(...);
```

Only runs when the line got a way.

Same loop: answer everyone who asked.

No way, so nothing to update.

• **Why**: the given code sends data up only after installing the line. A bypass installs nothing, so the L2C waits forever: `DEADLOCK!`

Step 1: the LLC keeps no copy from DRAM

```
// handle_fill(), after the way is chosen
if (EXCLUSIVE_LLC && NAME == "LLC")
  way = NUM_WAY;    // bypass

bool success = filllike_miss(set, way, *fill_mshr);
```

This code runs in every cache.

NUM_WAY = allocate nothing.

- **Why**: the L2C is about to install this line. An LLC copy would be a duplicate.

Step 2: the L2C sends every victim down

```
// filllike_miss()
bool evicting_dirty = ... && fill_block.dirty;
if (EXCLUSIVE_LLC && NAME == "cpu0_L2C")
  evicting_dirty = fill_block.valid;

if (evicting_dirty) {
  ...
  writeback_packet.dirty = fill_block.dirty;
  lower_level->add_wq(&writeback_packet);
}
```

Today: only dirty victims go down.

An empty way sends nothing.

task3.patch 3: clean stays clean.

- **Why**: after step 2, the LLC gets lines only from L2C victims. Clean victims must go down too.

Step 3: an LLC hit moves the line up

```
// readlike_hit()
if (EXCLUSIVE_LLC && NAME == "LLC") {
  handle_pkt.dirty = hit_block.dirty;
  hit_block.valid = false;
  hit_block.dirty = false;
}

for (auto ret : handle_pkt.to_return)
  ret->return_data(&handle_pkt);
```

The dirty bit goes up.

The LLC drops its copy.

Never write back a dropped line.

- **Why**: the L2C installs the line when this response arrives, so the LLC copy must go.

Checking your steps*

after (make task3, soplex)	L2C lines also in LLC	IPC	L2C MPKI	LLC MPKI
`given cache.cc`	8191	0.317071	35.6121	28.6951
`step 1`	8191	0.317071	35.6121	28.6951
`step 2`	5800	0.383673	35.6123	30.1266
`step 3`	3899	0.325360	35.6121	54.0824
`step 4`	0	0.334986	35.6120	61.6739

- Step 1 alone changes nothing: nobody bypasses yet.
- `make check-task3` checks IPC, L2C MPKI and LLC MPKI, and prints the count.

* Can you “prove” that the 3 steps are sufficient for maintaining exclusivity? :)

Results: long runs (20M + 25M)

trace	IPC NINE	IPC exclusive	change
soplex	0.3233	0.3433	+6.2%
mcf	0.4092	0.4185	+2.3%
bwaves	0.2436	0.2451	+0.6%
gcc	0.1138	0.1138	0.0%

LLC MPKI doubles!!!: every L2C victim is now an LLC write; LLC is now operating on a different (and probably less reuse-friendly) dataset.

We got a speed up on soplex and mcf: more LLC hits, less DRAM traffic per load.

IMP Q: There's always a tradeoff. What is the hardware/power cost in this case???

OPT Q: Why do gcc and bwaves gain nothing?

A: remember cache fitting vs thrashing vs streaming… soplex/mcf were thrashing at LLC… what about gcc, bwaves? Read MPKI!!!

OPTIONAL Questions: when does an exclusive LLC help?

Speedup in general

Q1. A load misses in the L2C. What decides how long it waits?

Q2. Which misses can a bigger cache remove?

Speedup from exclusivity

Q3. How does an exclusive LLC remove misses?

Q4. Which programs gain, and which do not?

Q5. What does exclusivity cost?

Q6. Why not make the L2C exclusive of the L1D instead?

Clean and dirty lines

Q7. What if every victim arrives in the LLC marked dirty?* Note 1 in a previous slide

Q8. What if an LLC hit does not send the dirty bit up?

Q9. What if a dirty line stays in the LLC on a hit, and only clean lines move up?

If you find a new term here, google it or ask TAs. These are not compulsory. Totally optional content.

Answers (1/2)

Q1. A load misses in the L2C. What decides how long it waits?

Where the line is found. An LLC hit costs about 20 cycles; DRAM costs a few hundred (soplex: 422 on average). This is the miss penalty in the average memory access time (AMAT).

Q2. Which misses can a bigger cache remove?

Capacity misses. Compulsory misses (first touch) and coherence misses (other cores) stay.

Q3. How does an exclusive LLC remove misses?

No copies of L2C lines, so the effective capacity is 2.5 MB instead of about 2 MB. Fewer capacity misses go to DRAM.

Q4. Which programs gain, and which do not?

Programs that reuse lines just beyond what fits: soplex +6.2%, mcf +2.3%. Streaming programs rarely hit in the LLC: gcc 0%, bwaves +0.6%.

Q5. What does exclusivity cost?

Every L2C victim moves to the LLC: more on-chip traffic (soplex LLC MPKI 27.7 to 59.0). In a multicore, an inclusive LLC also filters coherence snoops; an exclusive one cannot so further time lost (note that this will NOT show up as MPKI cleanly; this is another example of why just MPKI/MissRate are incomplete metrics).

Answers (2/2)

Q6. Why not make the L2C exclusive of the L1D instead?

The L1D adds only 48 KB to 512 KB (+9%), and an extra L2C hit saves an LLC access, not a DRAM access.

Q7. What if every victim arrives in the LLC marked dirty?

Every LLC eviction becomes a DRAM write. Too much DRAM traffic. On soplex, DRAM write traffic goes from 266k to 621k writes and IPC falls from 0.323 to 0.279.

Q8. What if an LLC hit does not send the dirty bit up?

The L2C stores a dirty line as clean, so the write is lost. DRAM writes fall to 194k, below NINE's 266k: faster, but wrong.

Q9. What if a dirty line stays in the LLC on a hit, and only clean lines move up?

It works without an upward dirty bit, but dirty lines are in both caches (1481 on soplex). IPC is 0.340 instead of 0.343.